coreclaw-guide

AI for Science — AI エージェントが必須の時代

科学研究における AI の役割

科学研究のあらゆるフェーズ — 仮説立案、実験計画、データ解析、論文執筆 — に AI を統合的に活用する **AI for Science** が世界的に加速しています。

従来の研究	AI for Science
ツールの分散（個別操作）	エージェントが適切なツールを自動選択・実行
解析の属人化	スキルとして体系化し再利用可能
再現性の手順書記述漏れ	コンテナ環境で実行環境を完全に固定
学際的な壁（他分野ツール習得に時間）	自然言語で指示するだけで専門ツールを活用

日本でも **文部科学省** が「AI for Science 科学研究革新プログラム」を推進しており、AI を活用した科学研究の加速は国家戦略として位置づけられています。JST の AIP ネットワークラボでは、革新的な人工知能基盤技術の研究開発が進められ、ライフサイエンス、材料科学、数理科学など多分野で AI エージェントの活用が広がっています。

こうした潮流の中で、**科学研究を含むあらゆるドメインで AI エージェントを安全に運用できる基盤** が必要とされています。

AI エージェントの爆発的普及とセキュリティ問題

OpenClaw — 史上最速で普及した AI エージェント

2025 年 11 月、オーストリアの開発者 Peter Steinberger が個人プロジェクトとして公開した AI エージェント **OpenClaw**（旧 Clawdbot → Moltbot）は、わずか数ヶ月で爆発的に普及しました。

- **GitHub**: 247,000 スター、47,700 フォーク（2026 年 3 月時点）
- **Cloudflare 株価**: OpenClaw の話題で **14% 急騰**（ローカル実行インフラとして注目）
- **Mac mini 品薄**: OpenClaw を自宅サーバーとして常時稼働させるユーザーが急増し、Apple Mac mini M4 が各地で売り切れに
- **グローバル展開**: Silicon Valley から中国まで、Alibaba・Tencent・ByteDance が自社プラットフォームに統合
- **OpenAI が獲得**: 2026 年 2 月、Steinberger は OpenAI に参画を発表

OpenClaw は「**AI that actually does things**（AI が実際に物事をやる）」をキャッチフレーズに、メール管理、カレンダー操作、Web ブラウジング、ショッピングなどをメッセージングアプリ（WhatsApp, Telegram, Discord）経由で自律的に実行します。

セキュリティ上の深刻な問題

しかし、OpenClaw の急速な普及は深刻なセキュリティリスクを露呈しました。

問題	詳細
ホストへの直接アクセス	エージェントがホスト OS 上で任意のコマンドを実行可能
プロンプトインジェクション	悪意ある WhatsApp メッセージ等でエージェントを乗っ取り可能
データ流出	Cisco のセキュリティチームがサードパーティスキルによるデータ窃取を実証
MoltMatch 事件	エージェントがユーザーの指示なしに出会い系サイトにプロフィールを作成
中国政府が規制	2026 年 3 月、国有企業と政府機関での OpenClaw 使用を制限

Palo Alto Networks は、OpenClaw が「**致命的な三重リスク**（lethal trifecta）」— プライベートデータへのアクセス、信頼できないコンテンツへの露出、外部通信能力 — を持つと警告しました。

:::note alert OpenClaw のメンテナー自身が Discord で警告しています：「コマンドラインの使い方がわからないなら、このプロジェクトはあなたには危険すぎる」 :::

NanoClaw — コンテナ分離で安全性を確保

OpenClaw のセキュリティ問題に対するアンチテーゼとして、2026 年初頭に **NanoClaw** が登場しました。Qwibit.ai チームが開発し、**Anthropic（Claude の開発元）が支援** したプロジェクトです。

NanoClaw の最大の革新は **OS レベルのコンテナ分離** です。

	OpenClaw	NanoClaw
コードベース	約 400,000 行以上	約 4,000 行（30 分で監査可能）
セキュリティ	アプリレベル	OS レベル（Docker / Apple Containers）
依存パッケージ	70 以上	10 未満
セットアップ	1〜2 時間	数分
エンジン	75+ LLM プロバイダ対応	Claude Agent SDK

NanoClaw は「コードベースが小さく監査しやすい」「コンテナ分離でエージェントがホストを汚染しない」「依存関係が少なくサプライチェーンリスクが低い」という点で、エンタープライズ用途に適したアプローチを示しました。

CoreClaw — AI for Science のための AI Agent OS

なぜ CoreClaw を開発したのか

CoreClaw は **AI for Science のための AI Agent OS** として開発しました。その背景には、科学研究固有の 2 つの要件があります。

1. コンテナ分離の必要性

AI for Science では、エージェントが実験データの分析のために **プログラムを自律的に開発・実行** します。Python スクリプトの生成、ライブラリのインストール、データの可視化、統計処理など、ホスト環境で任意のコマンドを実行する必要があるため、OpenClaw のようにホスト上で直接実行するのは危険です。NanoClaw が示したコンテナ分離は、まさにこの問題の解決策でした。

2. タスクごとの LLM 自動選択

科学研究ではタスクごとに最適な LLM モデルが異なります。コード生成は Claude、データ解析は GPT、文献検索は Gemini など、単一のモデルに縛られることは実用上の大きな制約です。NanoClaw の **Claude Agent SDK 限定** という制約が、科学研究用途では障壁となりました。

CoreClaw はこれらの要件を踏まえ、NanoClaw のコンテナ分離思想を継承しつつ以下の課題を解決しました。

NanoClaw の制約	CoreClaw の解決策
Claude Agent SDK 限定	GitHub Copilot CLI（Auto モードでタスクに応じた LLM 自動選択）
スキルシステムなし	プラグイン型 Agent Skills（`skills/` にファイルを置くだけ）
用途がメッセージング中心	ChatGPT スタイル Web UI + 実験管理 + 成果物管理
科学ツール連携なし	MCP サーバー連携（ToolUniverse 1000+ 科学ツール等）
日本語非対応	CJK フォント内蔵、日本語グラフ・図表生成対応

一言で言えば: AI for Science のために開発された、SKILL.md ファイルを置くだけでどんなドメインの AI エージェントにもなれるセキュアなプラットフォーム。

:::note warn - **想定読者**: AI エージェントの環境構築に興味があるエンジニア・研究者 - **所要時間**: 約 15 分 - **難易度**: 中級（Docker, Node.js の基本知識があること） - **検証環境**: Windows 11 + WSL2 (Ubuntu) / macOS :::

CoreClaw の全体像

コンセプト

CoreClaw は、AI for Science のための AI Agent OS として開発された、GitHub Copilot CLI を AI エンジンとして Docker コンテナ内で安全に実行するプラットフォームです。実験データの分析プログラムをエージェントが自律的に開発・実行するためのコンテナ分離と、タスクに応じた LLM 自動選択を備え、プラグイン型の Agent Skills でコンサルティング、教育、科学研究など任意のドメインに対応できます。

```
┌──────────────────────────────────────────────────────┐
│  CoreClaw = Chat Frontend + Agent Skills Engine      │
│                                                      │
│  ┌──────────┐  ┌──────────────────────────┐         │
│  │  Web UI  │  │  Agent Skills（プラグイン）  │       │
│  │  (SPA)   │  │  skills/                  │        │
│  └────┬─────┘  │  ├── consultant/          │        │
│       │        │  ├── educationalist/      │        │
│       │ WS     │  ├── scientist/           │        │
│  ┌────▼─────┐  │  └── your-custom-skill/   │        │
│  │  Server  │  └──────────────────────────┘         │
│  │  (Node.js) │                                      │
│  └────┬─────┘                                        │
│       │ Docker                                       │
│  ┌────▼──────────────────────────────────┐           │
│  │  coreclaw-agent Container             │           │
│  │  ├── GitHub Copilot CLI (AI エンジン)  │           │
│  │  ├── Python 3 + uv (MCP サーバー)      │           │
│  │  └── .github/skills/ (自動同期)        │           │
│  └──────────────────────────────────────┘           │
└──────────────────────────────────────────────────────┘
```

主要機能

- 🐳 **Docker サンドボックス** — 各タスクが独立コンテナで安全に実行
- 🔌 **Agent Skills プラグイン** — `skills/` に SKILL.md を置くだけで拡張
- 💬 **ChatGPT スタイル Web UI** — 2 ペイン SPA、マークダウン対応、ファイルアップロード
- 🔗 **MCP サーバー連携** — ToolUniverse, Deep Research, カスタムサーバー
- 🔄 **GitHub 同期** — 実験結果を GitHub リポジトリに push/pull

- 📌 **チャットグループ管理** — ピン留め、検索、グループごとのスキル・MCP 設定
- 🔐 **認証プロキシ** — GitHub トークンをコンテナに渡さず安全に認証
- 🎤 **音声入力** — Whisper 対応の音声認識
- 🔍 **スキルスキャナー** — セキュリティスキャン + ホワイトリスト管理
- ⬆️ **セルフアップデート** — git pull ベースの自動更新

Agent Skills — プラグイン型スキルシステム

仕組み

CoreClaw の最大の特徴は、Agent Skills をプラグインとして扱えることです。Agent Skills の仕様は、GitHub Copilot をはじめ様々な AI システムで採用されている **Open Standard** です。詳細は Agent Skills 仕様 を参照してください。

```
skills/                          # ← ここにスキルを置くだけ
├── consultant/                  # コンサルティング（53 フレームワーク）
│   ├── SKILL.md                 # スキル定義（YAML + Markdown）
│   ├── prompts/                 # プロンプトテンプレート
│   └── skills/                  # サブスキル
├── educationalist/              # 教育（175 教育理論）
│   ├── SKILL.md
│   ├── data/                    # 教育理論 DB（SQLite + JSON）
│   └── skills/                  # 10 サブスキル
├── scientist/                   # 科学（195 サブスキル）
│   ├── SKILL.md
│   └── skills/
│       ├── scientific-hypothesis-pipeline/
│       ├── scientific-protein-structure-analysis/
│       ├── scientific-drug-target-profiling/
│       └── ... (195 サブスキル)
└── general-assistant/           # 汎用アシスタント
    └── SKILL.md
```

スキル読み込みフロー

```
1. skills/ ディレクトリをスキャン
2. 各スキルの SKILL.md から YAML フロントマターを解析
3. チャットグループにスキルを割り当て
4. タスク実行時にスキルをコンテナの .github/skills/ に同期
5. GitHub Copilot CLI が自動検出して利用
```

同梱スキルパッケージ

サンプルとして 8 つのスキルパッケージを同梱しています。

スキル	内容	特徴
consultant	汎用コンサルティング	SHIKIGAMI メソドロジー、53 フレームワーク、4 フェーズワークフロー
consultant-acn	Accenture スタイル	デジタルトランスフォーメーション、テクノロジー戦略
consultant-bcg	BCG スタイル	成長戦略、コスト構造改革
consultant-mck	McKinsey スタイル	MECE、ピラミッドストラクチャー
consultant-pwc	PwC スタイル	リスクアドバイザリー、規制対応
educationalist	教育アシスタント	175 教育理論 DB、10 サブスキル、カリキュラム設計
general-assistant	汎用アシスタント	一般的なタスク処理
scientist	科学アシスタント	195 サブスキル内蔵（創薬、ゲノミクス、臨床等）

カスタムスキルの作成

独自のスキルを作成するのは非常に簡単です。

1. ディレクトリを作成

```
mkdir skills/my-skill
```

2. SKILL.md を作成

```
---
name: my-skill
description: |
 カスタムスキルの説明。
 このスキルがどんなタスクに適しているかを記述します。
---

# My Custom Skill

## 役割
あなたは○○の専門家です。

## 指示
- ユーザーの質問に対して、○○の観点から回答してください。
```

```
- 必要に応じて、△△のフレームワークを使用してください。

## 出力形式
- Markdown 形式でレポートを出力してください。
```

3. チャットグループに割り当て

Web UI のサイドバーから新しいチャットグループを作成し、スキルセレクターから `my-skill` を選択するだけです。

:::note info SKILL.md の YAML フロントマターで `name` と `description` を定義するだけで、CoreClaw が自動的にスキルとして認識します。サブスキルを追加する場合は `skills/` サブディレクトリにさらに SKILL.md を配置します。 :::

ZIP アップロードによるスキル登録

スキルを手動でディレクトリに配置する以外に、**ZIP ファイルをアップロード** して登録することもできます。

Settings > Skills タブの「📦 Upload ZIP」ボタンから ZIP ファイルを選択するだけで、スキルが自動的に展開・登録されます。

```
my-skill.zip
├── SKILL.md          # 必須：スキル定義ファイル
├── prompts/          # オプション：プロンプトテンプレート
├── data/             # オプション：データファイル
└── skills/           # オプション：サブスキル
```

ZIP ファイルの要件：

- **SKILL.md が必須** — ZIP 内に `SKILL.md` が含まれていない場合はエラーになります
- **トップレベルフォルダ自動検出** — ZIP が単一のフォルダを含む場合、そのフォルダの中身をスキルとして展開します
- **既存スキルの上書き** — 同名のスキルが既に存在する場合は置き換えられます

:::note info コミュニティやチーム内でスキルを共有する際に、ZIP ファイルでの配布が便利です。スキルのエクスポート（ZIP ダウンロード）にも対応しています。 :::

スキルスキャナー — セキュリティスキャン

サードパーティのスキルを導入する際に、安全性を確認するための **スキルスキャナー** 機能を搭載しています。

スキャンの実行

Settings > Skills タブで各スキルの横にあるスキャンバッジ（●）をクリックすると、スキルの全テキストファイルに対してセキュリティスキャンが実行されます。

スキャン結果は 3 段階のステータスで表示されます：

ステータス	色	意味
Safe	🟢 緑	検出なし
Caution	🟡 黄	中リスクのパターンを検出
Warning	🔴 赤	高リスクのパターンを検出

検出パターン

高リスク（赤） — 悪意ある操作の可能性：

- `rm -rf` / 破壊的な削除コマンド
- `curl ... | bash` — リモートコード実行
- `eval()` / `new Function()` — 動的コード実行
- `subprocess` / `child_process` — シェルコマンド実行
- `sudo` — 権限昇格
- `DROP TABLE` / `DELETE FROM` — 破壊的 SQL
- 認証情報の環境変数からの読み取り

中リスク（黄） — 注意が必要な操作：

- 外部 HTTP リクエスト（`fetch`, `requests.get` 等）
- ハードコードされた URL
- 環境変数アクセス
- システムモジュールのインポート
- ファイル書き込み・削除操作
- パストラバーサル（`../`）

ホワイトリスト管理

スキャン結果の各検出項目には **OK チェックボックス** が付いており、確認済みの検出をホワイトリストに登録できます。ホワイトリストに登録された項目はステータス計算から除外されるため、既知の安全なパターンを除外して本当に注意すべき項目だけに集中できます。

:::note alert スキルスキャナーは簡易的なヒューリスティック検出であり、完全なセキュリティ保証を提供するものではありません。**必ず、各自がスキルの SKILL.md やプロンプトの内容を確認したうえで使用してください。** スキャン結果はあくまで参考情報です。 :::

アーキテクチャ

システム全体像

```
┌──────────────────────────────────────────────────────┐
│  Browser (SPA)                                       │
│  http://localhost:3000                               │
└──────────────┬───────────────────────────────────────┘
               │ HTTP / WebSocket
┌──────────────▼───────────────────────────────────────┐
│  web-server.ts (API + WebSocket サーバー)              │
│  ├── Settings, Skills, Experiments, Scanner APIs      │
│  ├── Check/Update (CoreClaw + Copilot CLI)            │
│  └── GitHub Sync                                      │
├──────────────────────────────────────────────────────┤
│  container-runner.ts                                  │
│  ├── Docker spawn + volume mounts                     │
│  ├── Skills sync (skills/ → .github/skills/)          │
│  ├── MCP config injection (--additional-mcp-config)   │
│  └── GitHub MCP tools (--enable-all-github-mcp-tools) │
├──────────────────────────────────────────────────────┤
│  credential-proxy.ts (:3001)                          │
│  └── GitHub トークンの安全な注入                        │
└──────────────┬───────────────────────────────────────┘
               │ Docker
┌──────────────▼───────────────────────────────────────┐
│  coreclaw-agent:latest                                │
│  ├── agent-runner (TypeScript)                        │
│  │   └── copilot -p <prompt> --allow-all              │
│  ├── GitHub Copilot CLI (@github/copilot)             │
│  ├── Python 3 + uv (MCP サーバー実行)                  │
│  ├── Chromium (Web 自動化)                             │
```

```
│   └── CJK フォント（日本語対応）                                  │
└──────────────────────────────────────────────────────────────────┘
```

コンテナ内部

`coreclaw-agent:latest` Docker イメージの主要コンポーネント:

コンポーネント	用途
Node.js 22-slim	ランタイム
GitHub Copilot CLI	AI エンジン（Claude, GPT, Gemini 等を自動選択）
Python 3 + uv	MCP サーバー実行、データサイエンスタスク
Chromium	Web 自動化
CJK フォント	日本語グラフ・図表の生成
matplotlib（日本語設定済）	可視化
Git, curl, sqlite3	ユーティリティ

セキュリティモデル（多層防御）

CoreClaw は多層防御モデルを採用しています。

レイヤー	内容
コンテナ分離	各エージェントが専用 Docker コンテナで実行（`--rm` で自動削除）
認証プロキシ	GitHub トークンはコンテナに渡さず、プロキシ経由で注入
環境変数保護	`.env` はコンテナ内で `/dev/null` にシャドウマウント
ソースコード保護	プロジェクトディレクトリは読み取り専用でマウント
ファイル権限	設定ファイルは `chmod 600`
非 root 実行	コンテナ内は `node` ユーザーで実行
スキルスキャナー	サードパーティスキルの簡易セキュリティスキャン

:::note warn **既知の制限事項：** エージェントの作業ディレクトリ（`groups/`）やセッションデータ（`data/`）はホスト上に**読み書き可能**でマウントされています。これは入力プロンプト・生成物（Markdown, Python, PNG 等）を証跡としてホスト側に永続保存するための設計ですが、エージェントがホストのこれらのディレクトリに任意のファイルを書き込める点はリスクとなります。生成物の内容は必ず確認してから使用してください。 :::

認証プロキシの仕組み

```
┌──────────────┐     ┌──────────────────┐     ┌──────────────┐
│  Container   │     │ Credential Proxy │     │  GitHub API  │
│              │──►  │ :3001            │──►  │  api.github  │
│ (トークンなし)│     │ Authorization:   │     │  .com        │
│              │     │ Bearer $TOKEN    │     │              │
└──────────────┘     └──────────────────┘     └──────────────┘
```

コンテナは `GITHUB_API_URL` を認証プロキシに向けるだけ。プロキシがリクエストを `api.github.com` に転送する際に、`Authorization` ヘッダーに GitHub トークンを付与します。これにより、コンテナ内に一切のシークレットが存在しません。

MCP サーバー連携

CoreClaw は外部 MCP サーバーとの連携をサポートしています。

プリセット

サーバー	説明
ToolUniverse	ハーバード大学 Zitnik Lab の 1000+ 科学ツール（文献検索、タンパク質解析等）
Deep Research	構造化リサーチレポート生成

プリセットの追加方法

Settings > MCP Servers タブに「+ ToolUniverse」「+ Deep Research」ボタンがあります。クリックするだけでプリセット設定が自動入力されます。

```
ToolUniverse:
  Type: stdio
  Command: uvx
  Args: tooluniverse
  Env: PYTHONIOENCODING=utf-8

Deep Research:
  Type: stdio
  Command: uvx
  Args: mcp-server-deep-research
```

追加後、Save ボタンで保存すれば完了です。次回のエージェント実行時から MCP サーバーが利用可能になります。

:::note info `uvx` は Python パッケージランナーです。コンテナ内に Python 3 + uv がプリインストールされているため、追加のインストールは不要です。なお、CoreClaw にこれらのコードはバンドルされておらず、実行時に `uvx` が動的にダウンロードします。ToolUniverse は Apache-2.0 ライセンス、Deep Research は MIT ライセンスで、いずれもオープンソースです。 :::

カスタム MCP サーバー

Settings UI から `stdio`（コマンド実行型）や `sse/streamable-http`（URL 接続型）の MCP サーバーを追加できます。設定は `COPILOT_MCP_CONFIG` 環境変数として Docker コンテナに注入されます。

GitHub MCP Tools

GitHub API へのアクセスも MCP 経由で提供されます。repos, issues, PRs 等の操作が可能です。

環境構築

前提条件

- Node.js 22+
- Docker
- GitHub Copilot ライセンス（Individual, Business, or Enterprise）
- GitHub Token

セットアップ手順

1. リポジトリのクローン

```
git clone https://github.com/nahisaho/coreclaw.git
cd coreclaw
```

2. 依存関係のインストール

```
npm install
```

3. 環境変数の設定

```
cp .env.example .env
```

`.env` ファイルを編集して `GITHUB_TOKEN` を設定します。GitHub CLI がインストール済みであれば:

```
echo "GITHUB_TOKEN=$(gh auth token)" > .env
```

4. TypeScript ビルド

```
npm run build
```

5. エージェントコンテナのビルド

```
./container/build.sh
```

6. 起動

```
npm start
```

ブラウザで http://localhost:3000 を開きます。

カスタムポートで起動

```
CORECLAW_WEB_PORT=3050 CREDENTIAL_PROXY_PORT=3051 npm start
```

開発モード（ホットリロード）

```
npm run dev
```

設定オプション

変数	デフォルト	説明
`GITHUB_TOKEN`	(必須)	GitHub トークン（Copilot CLI 認証用）

変数	デフォルト	説明
`CORECLAW_WEB_PORT`	`3000`	Web サーバーポート
`CREDENTIAL_PROXY_PORT`	`3001`	認証プロキシポート
`ASSISTANT_NAME`	`Andy`	ボットのトリガー名
`CONTAINER_IMAGE`	`coreclaw-agent:latest`	Docker イメージ名
`CONTAINER_TIMEOUT`	`1800000`	エージェントタイムアウト（ms）
`MAX_CONCURRENT_CONTAINERS`	`5`	最大並行エージェント数

使い方

基本的なフロー

1. **チャットグループの作成** — サイドバーの「＋ New Chat Group」をクリック
2. **スキルの選択** — 作成時にスキルセレクターから割り当てるスキルを選択
3. **メッセージの送信** — テキストボックスにプロンプトを入力して送信
4. **リアルタイム応答** — WebSocket 経由でストリーミング表示
5. **成果物の確認** — エージェントが生成したファイルを確認・ダウンロード

使用例：コンサルティング分析

```
「日本のEV市場の成長戦略について、BCGスタイルで分析レポートを作成してください。
市場規模、主要プレイヤー、技術トレンド、規制環境を含めてください。」
```

consultant-bcg スキルを割り当てたチャットグループで上記を送信すると:

1. BCG の成長戦略フレームワークを適用
2. 市場データの収集と分析
3. 構造化されたレポートの生成
4. MECE に整理された提言

使用例：科学研究

scientist スキルを割り当て、MCP プリセットで ToolUniverse を有効化:

```
「EGFR変異を持つ非小細胞肺がんの薬剤耐性メカニズムについて調査してください。
最新の文献と構造データに基づいて分析をお願いします。」
```

scientist スキルには 195 の科学サブスキル（創薬、タンパク質解析、ゲノミクス、疫学等）が内蔵されており、本格的な科学研究タスクを実行できます。

使用例：教育カリキュラム設計

```
「高校生向けのデータサイエンス入門カリキュラム（全12回）を設計してください。
構成主義的アプローチとプロジェクトベース学習を取り入れてください。」
```

educationalist スキルは 175 の教育理論データベースを内蔵しており、理論に基づいたカリキュラム設計を支援します。

今後の開発計画

CoreClaw は以下の方向で開発を進めています。

1. 🔌 **スキルマーケットプレイス** — コミュニティによるスキル共有・インストール
2. 🧠 **MCP サーバー拡充** — Deep GraphRAG、日本学術論文検索（CiNii, J-STAGE）
3. 📊 **マルチモーダル対応** — 画像入力の強化
4. 🤖 **マルチエージェント** — 複数エージェントの協調実行

スキルの開発やフィードバックなど、コミュニティからの貢献を歓迎しています。

まとめ

OpenClaw の爆発的普及は AI エージェントの可能性を示すと同時に、セキュリティの重要性を浮き彫りにしました。NanoClaw が示したコンテナ分離のアプローチを発展させ、CoreClaw は **コンテナ分離による実行環境の隔離** と **プラグイン型 Agent Skills** を兼ね備えた汎用プラットフォームを実現しました。

```
OpenClaw（爆発的普及 → セキュリティ問題露呈）
  ↓
NanoClaw（コンテナ分離で安全性確保 → Claude SDK 限定）
  ↓
CoreClaw（コンテナ分離 + マルチ LLM + Agent Skills プラグイン）
```

SKILL.md を置くだけ で、コンサルティング、教育、科学研究、あるいは完全に独自のドメインに対応する AI エージェントを構築できます。

クイックスタート

```
git clone https://github.com/nahisaho/coreclaw.git
cd coreclaw && npm install
echo "GITHUB_TOKEN=$(gh auth token)" > .env
./container/build.sh
npm start
# → http://localhost:3000 を開くだけ
```

セットアップは 5 ステップ。Docker と GitHub Copilot ライセンスがあれば、すぐに始められます。

:::note info 生成物はホスト側に永続保存される設計のため、完全なサンドボックスではありません。エージェントの出力内容は確認してから使用してください。詳細は「セキュリティモデル（多層防御）」セクションを参照してください。 :::

参考資料

- CoreClaw GitHub
- GitHub Copilot CLI
- OpenClaw Wikipedia
- OpenClaw のセキュリティリスク（Cisco Blogs）
- ToolUniverse GitHub（MCP プリセットの科学ツール群）
- JST AIP ネットワークラボ（文部科学省 AI 研究基盤）